

UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

RELAZIONE CONCLUSIVA DI INGEGNERIA
DEL SOFTWARE

Sistema di gestione issue in progetti software

Authors:
Ubaldo Musto N86005304
Giandomenico Iameo
N86002856

Supervisor:
Prof. Porfirio TRAMONTANA
Co-Supervisor:
Prof. Bernardo BREVE

Versione 1.0
Prima versione del prodotto software, contenente le funzionalità di base per la gestione collaborativa di issue in progetti software

Dipartimento di Ingegneria Elettrica e delle Tecnologie dell'Informazione
Corso di Laurea Triennale in Informatica

7 novembre 2026

COPYRIGHT ©2025, BY UBALDO AND GIANDOMENICO
ALL RIGHTS RESERVED.

Indice

Acronimi e abbreviazioni **iii**

Lista delle definizioni **iv**

1 Specifica dei requisiti **2**
1.1 Scopo del documento dei requisiti 2
1.2 Scopo del prodotto 2
1.3 Riferimenti 3
1.4 Descrizione del resto del documento 3

2 Descrizione generale **5**
2.1 Prospettiva del prodotto 5
2.2 Funzioni del prodotto 5
2.2.1 Requisiti funzionali utente 5
2.2.2 Diagrammi di caso d'uso 6
2.3 Caratteristiche utente 10
2.4 Presupposti e dipendenze 10
2.4.1 Dipendenze tecnologiche 10
2.4.2 Dipendenze sull'ambiente dell'utente 11

3 Requisiti specifici **12**
3.1 Requisiti funzionali 12
3.1.1 Template di Cockburn 13
3.1.2 Prototipazione visuale via Mock-up 14
3.2 Requisiti non funzionali 15
3.2.1 Requisiti del prodotto 15
3.2.2 Requisiti organizzativi 16

4 Design del Software **18**
4.1 Diagramma delle classi di design 18
4.1.1 Front-end 18
4.1.2 Back-end 23
I ruoli come sottoclassi 23
Bug come sottoclasse di Issue 24
Stato, priorità e duplicati 24
Organizzazione dei service 24

DTO e validazione 25
Sicurezza 25
Gestione degli errori 26
Regole di business 26
Filtri e ordinamenti nel database 26
4.2 Schema di persistenza dei dati 26
4.2.1 Database e gestione dello schema 27
4.2.2 Schema ER 27
4.3 Report di qualità del codice 27
4.3.1 Make sure disabling Spring Security's CSRF protection is safe . . 28

5 Verifica e convalida **31**
5.1 Progettazione dei casi di test 31
5.2 Descrizione dei casi di testi in JUnit 31
5.2.1 Metodo `selectToggleFilter()` 32
5.2.2 Metodo `setDate()` 34
5.3 Testing lato back-end 37
5.3.1 Test plan: creazione di una issue 37
Obiettivo 37
Attori del test 37
Ambiente di testing 37
Criteri di superamento 40
5.3.2 Metodo `creaIssue()` 40
Criterio di copertura delle classi di equivalenza 45

Acronimi e abbreviazioni

API	**A**pplication **P**rogramming **I**nterface
CASE	**C**omputer **A**ided **S**oftware **E**ngineering
REST	**RE**presentational **S**tate **T**ransfer
ANSI	**A**merican **N**ational **S**tandards **I**nstitute
IEEE	**I**nstitute **E**lectrical **E**lectronics **E**ngineers
SRS	**S**oftware **R**equirements **S**pecification

Lista delle definizioni

Utente	Si riferisce a qualsiasi utente del sistema e può essere sia *interno* che *esterno*.
Template di Cockburn	È una rappresentazione completa di un caso d'uso. Descrive in modo dettagliato tutti i passaggi necessari all'esecuzione di un caso d'uso e include sezioni di supporto come *precondizioni* e *postcondizioni*.
Mock-up	È un *artefatto*, una rappresentazione visiva statica utilizzata per descrivere un sottoinsieme delle funzionalità del sistema. Il suo fine è puramente illustrativo o espositivo.
Issue	È una segnalazione di un problema che è stato riscontrato da un membro del team di sviluppo. In altre parole, è un modo per avvisare il team che qualcosa dovrebbe essere sistemato.
Feature	In Informatica, si riferisce a una funzionalità o capacità specifica che il software offre ai propri utilizzatori. Nel contesto del prodotto software preso in esame, è un tipo di segnalazione (*issue*) usata per suggerire una nuova funzionalità.
Bug	In Italiano anche detto *Baco*, nell'ambito della programmazione Informatica, è un'anomalia che porta a un malfunzionamento di un software, producendo un risultato inatteso o errato.
Stakeholder	Gli *stakeholder* (soggetti interessati o persone coinvolte) sono tutte quelle persone che hanno un interesse nei confronti di un'organizzazione e che influenzano e/o sono influenzate dalle sue attività.
Caso d'uso	Si riferisce alla descrizione del comportamento del sistema quando riceve uno stimolo da parte di un attore. In UML, un caso d'uso è graficamente rappresentato da un ovale.
Attore	Specifica un ruolo (o un insieme di ruoli) assunto da un utente o altra entità che interagisce con il sistema.
Container	È una tecnologia che consente di creare un ambiente isolato, simulando un ambiente operativo, in cui le applicazioni possono essere eseguite senza preoccuparsi dei conflitti con le altre.

Linguaggi Object-Orientend	Sono linguaggi appartenenti al paradigma computazionale *Object-Oriented*. Questo pone al centro delle attività la modellazione di concetti del mondo reale e le loro relazioni che definiscono il problema che si vuole risolvere.
CASE	Significa *Computer-Aided Software Engineering* (ingegneria del software assistita dal computer) e comprende una vasta gamma di programmi diversi per aiutare le attività di processo del software.
Back-end	È la parte di un sistema informatico che gestisce il funzionamento di un'applicazione o sito web. Comprende tutto ciò che avviene dietro le quinte: elaborazione dei dati, logica del sistema, comunicazione della base di dati.
Front-end	È la parte di un'applicazione o di un sito web visibile e accessibile all'utente, ovvero l'interfaccia con cui l'utente interagisce.
Docker	È una delle piattaforme software container più utilizzate e diffuse nell'ambito della virtualizzazione a livello di sistema operativo.

Capitolo 1

Specifica dei requisiti

1.1 Scopo del documento dei requisiti

Il documento *SRS*, riportato nei primi tre capitoli, si propone di accogliere in forma organica una descrizione di tutte le specifiche tecniche e funzionali che caratterizzano il sistema per la gestione di issue in progetti software.
Esso è stato redatto seguendo le indicazioni dello standard IEEE/ANSI 830-1998 (IEEE 1998) per i documenti dei requisiti.

1.2 Scopo del prodotto

Il prodotto software che si vuole realizzare prende il nome di BugBoard26, una piattaforma che consentirà a un team di sviluppo di segnalare problemi relativi a un progetto, monitorare lo stato, assegnarli a membri del team e tenere traccia delle attività di risoluzione. Il sistema deve fornire le seguenti funzionalità:

1. Il sistema deve fornire un meccanismo di autenticazione semplice e sicuro, basato su email e password. Le informazioni gestite dall'applicazione sono critiche per l'azienda, ed è fondamentale preservarne l'integrità e la segretezza. Il sistema viene fornito con un account da amministratore già attivo, con credenziali di default. Un amministratore può creare ulteriori utenze, specificando una email, una password, e indicando se quell'utenza sarà "normale" oppure "di amministrazione".

2. Tutti gli utenti autenticati possono segnalare una issue indicando almeno un titolo e una descrizione. Alcuni utenti potrebbero voler specificare anche una priorità e sarebbe gradita la possibilità di allegare un'immagine. Le issue possono essere di diverso tipo: question (per richieste di chiarimenti), bug (per segnalare malfunzionamenti), documentation (per segnalare problemi relativi alla documentazione), e feature (per indicare la richiesta o il suggerimento di nuove funzionalità). Le issue create sono inizialmente nello stato "todo".

3. Il sistema deve offrire una vista riepilogativa delle issue, con la possibilità di filtrare o ordinare i risultati in base a criteri come tipologia, stato, priorità o altri parametri rilevanti.
4. Ogni bug dovrebbe poter essere assegnato (da un amministratore) a un membro del team. Quando viene assegnato un bug a un utente, quest'ultimo riceve una notifica. Gli utenti possono visualizzare i bug loro assegnati.
5. È necessaria una funzione di ricerca che consenta di trovare bug in base a parole chiave.
6. Il sistema dovrebbe suggerire automaticamente a chi assegnare un bug, basandosi sul carico di lavoro corrente degli utenti.

1.3 Riferimenti

Qui di seguito sono elencati i documenti e gli indirizzi web a cui fa riferimento la presente specifica dei requisiti, nonché i link al codice sorgente e al repository Docker

- Slide del corso di Ingegneria del software 2025/2026 fornite dai professori P.Tramontana e B.Breve.
- Ian, Sommerville. *Ingegneria del software*. 8ª edizione, Pearson Paravia Bruno Mondadori S.p.A., 2007.
- Modello IEEE/ANSI 830-1998 per strutturare il documento dei requisiti.
- Enciclopedia online Wikipedia.
- Codice sorgente Back-end.
- Codice sorgente Front-end.
- Link alla directory contenente l'immagine Docker e il file compose per il back-end

1.4 Descrizione del resto del documento

La restante parte del presente documento è suddivisa in due ulteriori capitoli che forniscono una descrizione sempre più dettagliata delle funzionalità del prodotto software in questione. Il capitolo immediatamente successivo, *Descrizione generale*, è composto dai seguenti paragrafi:

1. *Prospettiva del prodotto:* descrive il contesto o ambiente operativo in cui sarà utilizzato il prodotto software in questione. Verrà specificata l'architettura generale, se il prodotto è un membro di una famiglia di prodotti, un sostituto di altri sistemi oppure un prodotto completamente autonomo.

2. *Funzioni del prodotto:* illustra i principali servizi (o funzionalità) del prodotto corredati da diagrammi. Il livello di dettaglio usato in questo paragrafo è a un alto livello; i dettagli delle funzionalità saranno forniti del capitolo 3.

3. *Caratteristiche utente:* esplicità le conoscenze richieste per poter utilizzare il prodotto in questione e fornisce una descrizione generale degli utilizzatori previsti dal sistema.

4. *Presupposti e dipendenze:* illustra le condizioni che il sistema presume siano verificate al momento del suo avvio, durante la sua esecuziione e dopo la terminazione. Verranno specificati anche *elementi esterni* (altri sistemi, persone o componenti) da cui il prodotto dipende e che sono necessari per il suo corretto funzionamento.

Nel terzo capitolo, *Requisiti specifici*, verranno illustrate le funzionalità che il prodotto software realizza. Il capitolo è composto dai seguenti paragrafi:

1. *Requisiti funzionali:* fornisce una descrizione dettagliata dei casi d'uso che verranno presentati nel capitolo 2. Tali requisiti indicano come il sistema dovrebbe reagire a particolari input o come dovrebbe comportarsi in particolari situazioni.

2. *Requisiti non funzionali:* esplicita i vincoli sui servizi o sulle funzioni offerti dal sistema. In questa sezione, i requisiti imposti sono stati divisi secondo la *classificazione di Sommerville*.

Capitolo 2

Descrizione generale

2.1 Prospettiva del prodotto

Il prodotto software sarà un'applicazione basata su un'architettura a tre livelli, composta da due macro-componenti indipendenti: una parte di *back-end* e una di *front-end*.

Back-end
Avrà la responsabilità della gestione centralizzata dello stato del sistema, garantendo che tutte le informazioni e i dati persistenti siano conservati e aggiornati unicamente su di esso.

Front-end
Costituirà l'interfaccia utente che si appoggierà ai servizi REST esposti dal back-end.

2.2 Funzioni del prodotto

In questo paragrafo sono stati elencati i requisiti funzionali utente, dedotti dalle funzionalità descritte nel primo capitolo, e descritti mediante i diagrammi di caso d'uso. Il paragrafo è suddiviso in due parti:

Nella *prima parte* verrà fornito un elenco dei requisiti funzionali dedotti, espressi in modo comprensibile anche per gli utenti senza un'approfondita conoscenza tecnica.

Nella *seconda parte* verranno presentati i diagrammi di caso d'uso, i quali saranno associati a uno o più requisiti.

2.2.1 Requisiti funzionali utente

RF1 **Il sistema deve offrire un meccanismo di autenticazione.**

RF2 **Il sistema deve poter permettere a ogni account *Amministratore* di creare ulteriori utenti, sia normali che di tipo amministratore, specificando le loro credenziali temporanee di accesso.**

RF3 **Gli *utenti interni* possono segnalare issue, con la possibilità di specificare un titolo, una descrizione, una priorità, una immagine e il tipo.**

RF4 **Il sistema deve permettere la visualizzazione di tutte le issue segnalate, con la possibilità di filtrare e ordinare i risultati.**

RF5 **Il sistema deve permettere, solo a un *Amministratore*, di assegnare bug e notificare un membro del team.**

RF6 **Il sistema deve consentire a un *utente normale* di ricevere notifiche riguardanti i bug.**

RF7 **Ogni *utente normale* può visualizzare i propri bug assegnati.**

RF8 **Il sistema deve fornire una funzione di ricerca per trovare issue.**

RF9 **Il sistema dovrebbe suggerire automaticamente a chi assegnare un bug.**

2.2.2 Diagrammi di caso d'uso

Qui di seguito vengono illustrati i diagrammi di caso d'uso che modellano i requisiti sopra citati.

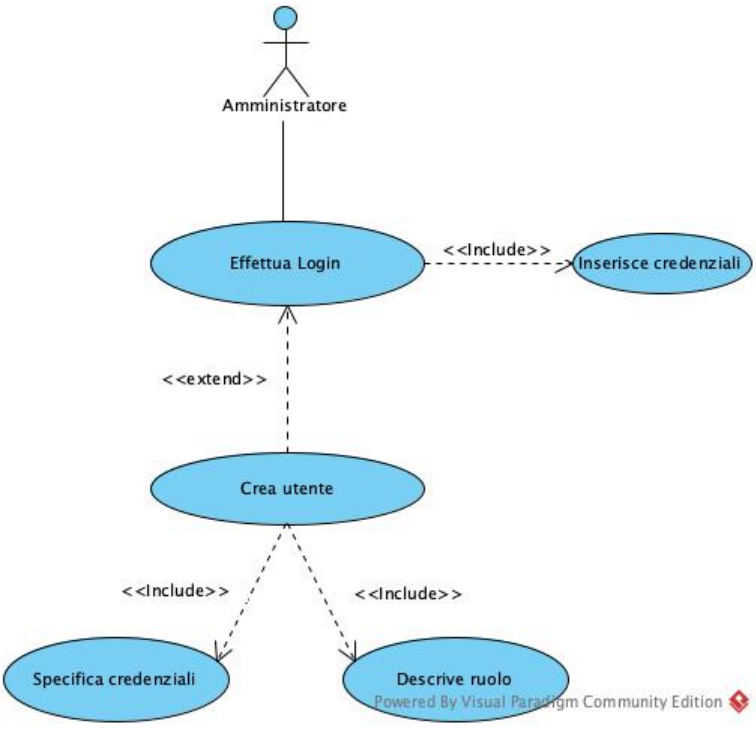


Figura 2.1: Modellazione requisiti *RF1*, *RF2*

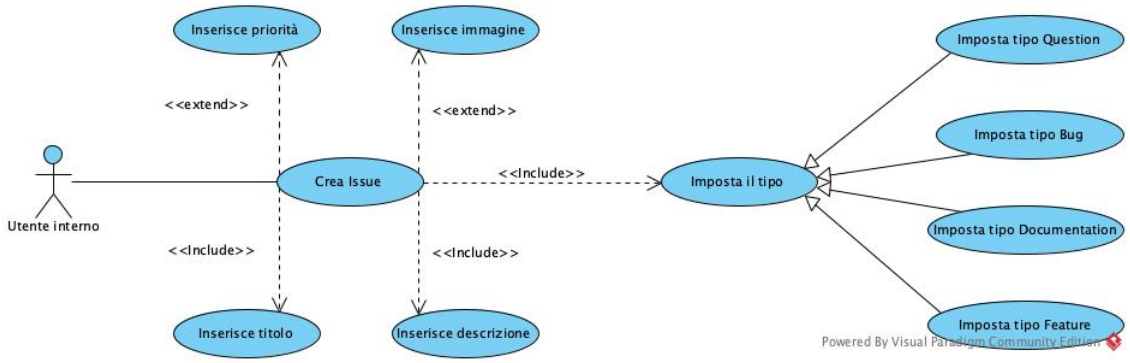


Figura 2.2: Modellazione requisito *RF3*

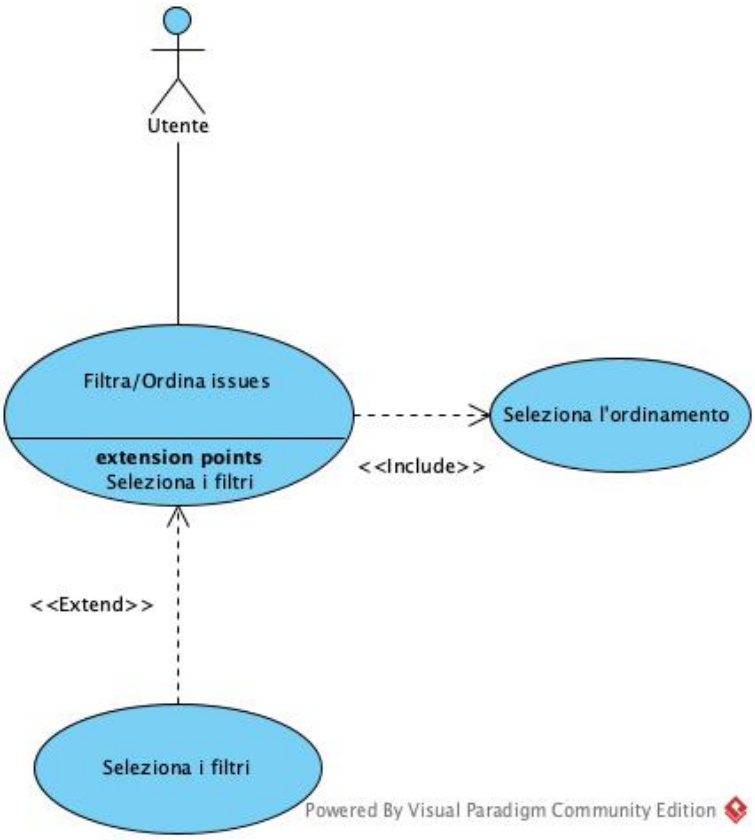


Figura 2.3: Modellazione requisito *RF4*

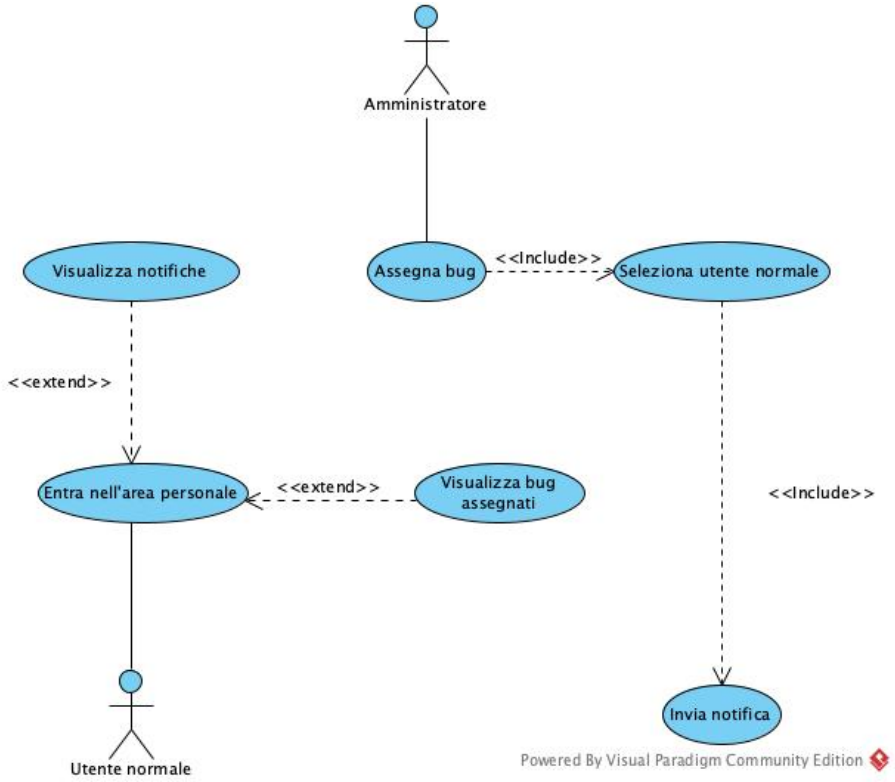


Figura 2.4: Modellazione requisiti *RF5*, *RF6*, *RF7*

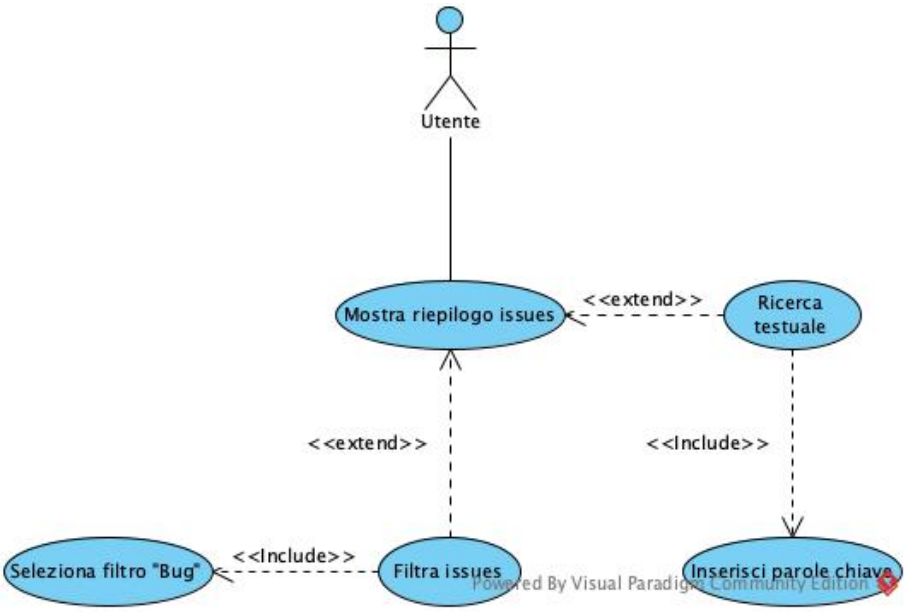


Figura 2.5: Modellazione requisiti *RF4*, *RF8*

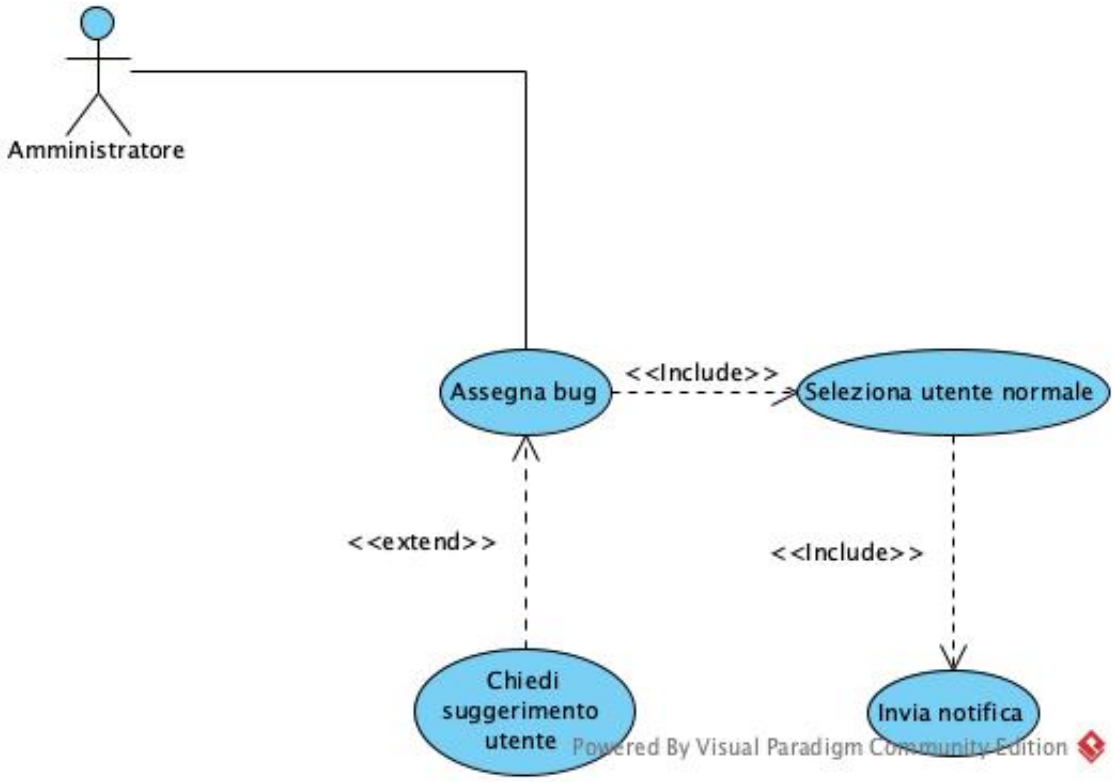


Figura 2.6: Modellazione requisiti *RF5,RF9*

2.3 Caratteristiche utente

L'utilizzo di questo prodotto software da parte degli utenti richiede che questi ultimi abbiano una buona conoscenza del dominio applicativo in cui il software opera. Per tale motivo, gli utenti del sistema sono stati divisi in due categorie in funzione del ruolo ricoperto.

1. *utente normale*
2. *utente mministratore*

Gli *utenti amministratori* sono coloro che utilizzano tutte le funzionalità messe a disposizione dal sistema.

Gli *utenti normali*, invece, utilizzano un sottoinsieme limitato delle funzionalità, pur possedendo una conoscenza approfondita del dominio applicativo. È il caso, in particolare, dei membri di un team di sviluppo. Questi devono essere in grado di riconoscere e saper descrivere chiaramente i loro problemi (bug, suggerimento di nuove funzionalità, ecc.).

Per qualsiasi tipologia di utente, una scarsa conoscenza del dominio applicativo può portare all'incapacità di riconoscere e descrivere correttamente tali problemi, con conseguente perdita di tempo e potenziali fraintendimenti. Nello specifico, le definizioni e i ruoli dei singoli attori del sistema sono:

Amministratore
Si riferisce a un utente che ha accesso completo al sistema, senza limitazione alcuna. Tra le operazioni che può effettuare figurano la gestione degli utenti e la gestione dei bug (come ad esempio, l'assegnazione di un bug a un membro del team).

Utente normale
Chiamato anche utente *regolare*, si riferisce a un utente senza privilegi speciali. Il suo ruolo è quello di segnalare problemi, proporre miglioramenti o fare richieste funzionali.

Tali account vengono assegnati normalmente ai membri di un team di sviluppo, i quali possono anche visualizzare i bug assegnati in un'apposita sezione.

2.4 Presupposti e dipendenze

2.4.1 Dipendenze tecnologiche

- *Containerizzazione del back-end*: Il committente desidera che una componente del software, ovvero la parte di *back-end*, sia distribuita utilizzando tecnologie di containerizzazione come *Docker*.
  - *Dipendenze*: Si assume che le macchine server supportino l'esecuzione di *container Docker*, includendo un demone *Docker* installato correttamente e configurato.

2.4.2 Dipendenze sull'ambiente dell'utente

- *Connettività di rete*: L'architettura di sistema prevede che il *front-end* comunichi con il *back-end* anche quando questo risiede in una rete esterna.
  - *Dipendenze*: Qualora le parti si trovino in due reti separate, si assume che gli utenti finali dispongano di una connessione Internet stabile e affidabile per poter utilizzare l'applicazione senza interruzioni. La qualità dell'esperienza utente dipenderà direttamente dalle prestazioni della rete.

Capitolo 3

Requisiti specifici

Il presente capitolo cerca di fornire una descrizione più dettagliata dei requisiti funzionali utente presentati nel capitolo 2, nonché dei requisiti non funzionali imposti.

Anche se, in teoria, i requisiti di sistema dovrebbero semplicemente descrivere i vincoli operativi e non come il sistema dovrebbe essere progettato, il livello di dettaglio adottato ha reso impossibile escludere tutte le informazioni sulla progettazione.

3.1 Requisiti funzionali

Il presente paragrafo si presta a descrivere in maniera più dettagliata i casi d'uso presentati nel precedente capitolo. Per la descrizione testuale di tali casi d'uso si è deciso di adottare un formato proposto da Alistar Cockburn e chiamato *template di Cockburn*. Il motivo di tale scelta è legato alla sua struttura tabellare, che facilita la comprensione delle interazioni tra attori e sistema, offrendo una rappresentazione esaustiva di tutti gli eventi.

Si fa presente al lettore che le tabelle seguenti contengono riferimenti a *mock-up*. Per una maggiore comprensione si raccomanda di consultare questi artefatti quando vengono citati. Questo paragrafo è suddiviso nei seguenti sottoparagrafi:

1. *Template di Cockburn*: Presentazione del *template di Cockburn* applicato a un *caso d'uso* significativo: *Filtra issues*.
2. *Prototipazione visuale via Mock-up*: I prototipi selezionati rispecchiano fedelmente l'interfaccia utente realizzata. Ciò ha permesso di individuare agevolmente sia le incongruenze nel design grafico, sia quelle relative alla logica operativa del sistema.

3.1.1 Template di Cockburn

Use case #1	*Filtra/Ordina issues*		
Goal in Context	*Consente all'utente di ordinare e filtrare le issue in base a diversi criteri*		
Preconditions	*L'utente deve possedere un proprio account*		
Success End Condition	*Il sistema restituisce i risultati della ricerca avanzata*		
Failed End Condition	*Il sistema non produce nessun risultato di ricerca*		
Primary Actor	Utente		
Trigger	*L'utente clicca sul pulsante "Cerca"*		
Main Scenario	**Step n.**	**Utente**	**Sistema**
	1		*Mostra M1*
	2	*Clicca sul pulsante "Cerca"*	
	3		*Mostra M2*
	4	*Clicca sul pulsante "Filtra"*	
	5		*Mostra M3*
	6	*Seleziona il tipo di filtro*	
	7		*Mostra M4*
	8	*Clicca sul pulsante "Apply"*	
	9		*Mostra M5*

3.1.2 Prototipazione visuale via Mock-up

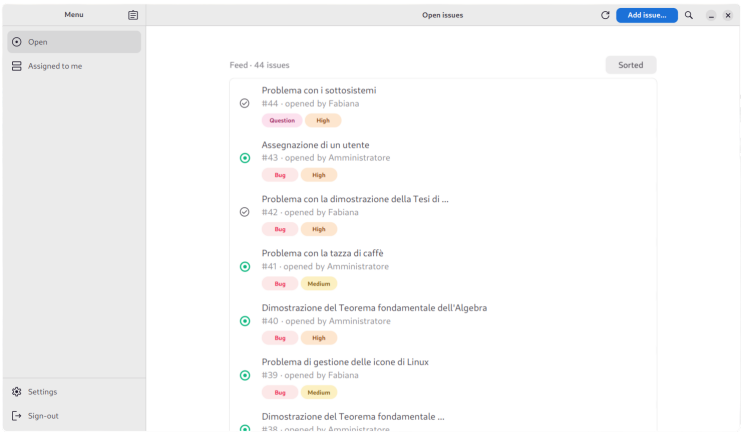


Figura 3.1: Mock-up *M1*

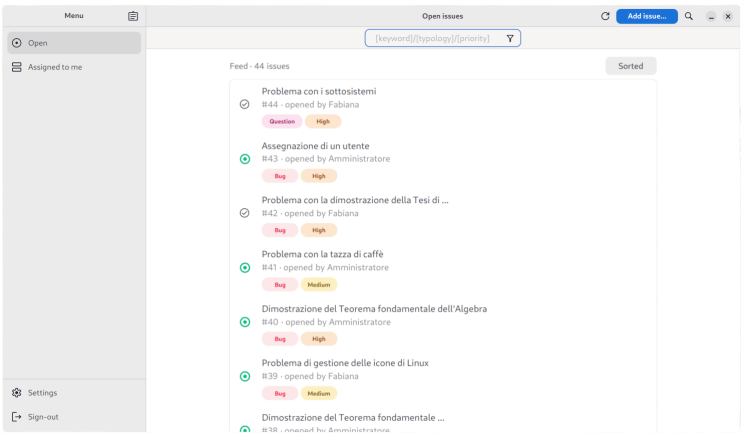


Figura 3.2: Mock-up *M2*

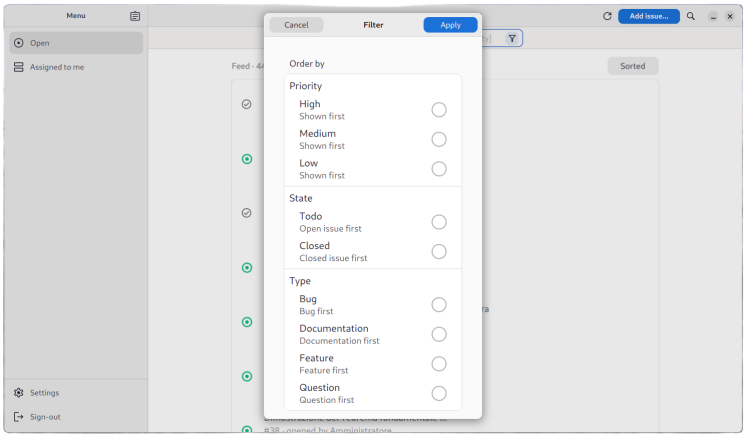


Figura 3.3: Mock-up *M3*

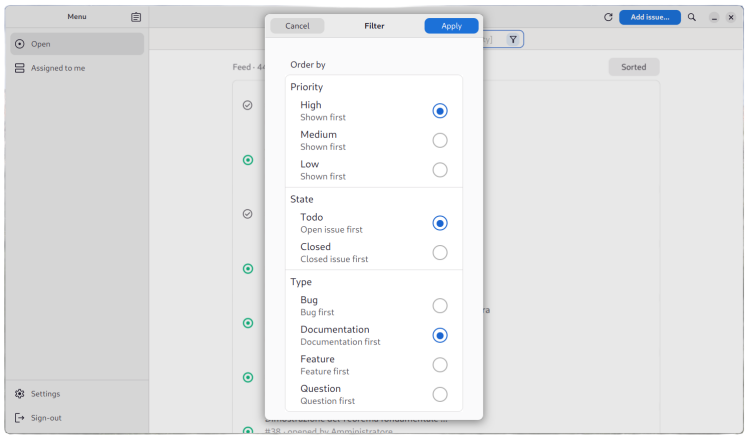


Figura 3.4: Mock-up *M4*

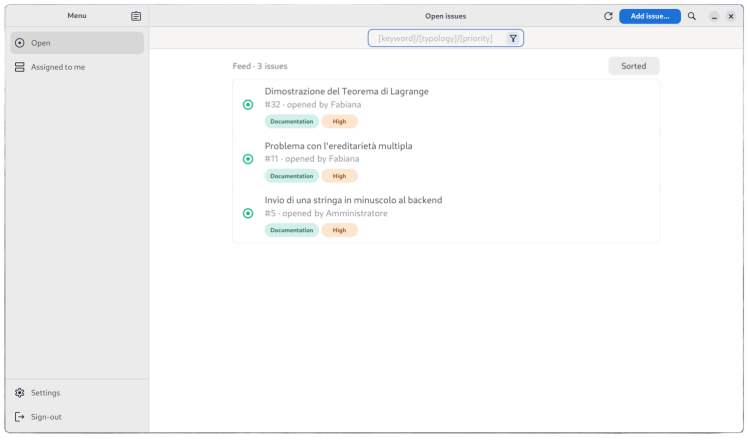


Figura 3.5: Mock-up *M5*

3.2 Requisiti non funzionali

3.2.1 Requisiti del prodotto

RP1 (*Requisito di affidabilità*): Il sistema deve offrire un alto grado di probabilità con cui fornisca correttamente i servizi come atteso dal cliente in un determinato periodo di tempo.

RP2 (*Requisito di apprendimento*): Il sistema deve offrire un alto grado di *familiarizzazione* dell'utente con le funzioni del prodotto entro un determinato periodo di tempo.

RP3 (*Requisito di adeguatezza*): Il sistema deve facilitare il raggiungimento dei compiti e degli obiettivi da parte dell'utente.

RP4 (*Requisito di portabilità*): Una parte del sistema (*back-end*) può essere eseguita in ambienti diversi. Il termine ambiente può riferirsi alla piattaforma hardware o all'ambiente software.

RP5 (*Requisito di confidenzialità*): Il sistema deve impedire accessi non autorizzati a informazioni riservate.

3.2.2 Requisiti organizzativi

RO1 (*Requisito sull'architettura*): Il sistema deve basarsi su una architettura distribuita.
Requisiti di sistema:

- Il sistema deve prevederre due macro-componenti indipendenti: *back-end* e *front-end*.
- Il *back-end* deve essere composto da almeno una componente che gestisce la logica di business e i dati.
- Il *back-end* deve offire una interfaccia di programmazione, con cui comunicare, mediante *API REST*.
- Il *front-end* deve comunicare con il *back-end* solo tramite le *API* esposte.
- Il *front-end* deve essere realizzato come applicazione desktop, come applicazione web, oppure come applicazione mobile.

RO2 (*Requisito di implementazione*): Per la fase di implementazione devono essere utilizzati linguaggi di programmazione Object-Oriented.
Requisiti di sistema:

- Sia il *back-end* che il *front-end* devono essere realizzati utilizzando linguaggi del paradigma ad oggetti.
- Il *front-end* dovrebbe essere un'applicazione desktop realizzata con Java e le librerie grafiche Swing o JavaFX.

RO3 (*Requisito sugli strumenti*): In fase di produzione devono essere utilizzati tool di CASE.
Requisiti di sistema:

- Strumenti come Visual Paradigm o Modelio devono essere utilizzati in fase di progettazione.

RO4 (*Requisito di consegna*): Il prodotto software e la relativa documentazione devono essere consegnati entro il 30 Settembre 2026.
Requisiti di sistema:

- Per il dettaglio completo delle scadenze di progetto si rimanda al capitolo **8** del documento **Progetto di Ingegneria del software.pdf**.

RO5 (*Requisito di unicità*): Il sistema deve essere progettato rispettando i requisiti antiplagio, in grado di soddisfare e superare gli standard richiesti in materia di originalità dei contenuti.
Requisiti di sistema:

- Tutti gli artefatti consegnati vengono processati con software antiplagio Turnitin, e raffrontati automaticamente con progetti degli anni scorsi e di quest'anno.

Capitolo 4

Design del Software

4.1 Diagramma delle classi di design

I diagrammi qui descritti sono quelli che effettivamente rappresentano il sistema sviluppato. Per una maggiore leggibilità, si è scelto di suddividere i diagrammi in base al sottosistema di appartenenza.

I diagrammi delle classi riportati afferiscono a un diverso livello di dettaglio: per la parte front-end si è scelto di dare maggiore importanza alle interazioni e ai flussi di comunicazione tra i diversi livelli dell'architettura, evitando di fare un "dump" speculare di tutto il codice sorgente e privilegiando solo la struttura logica, la separazione delle responsabilità e il flusso delle dipendenze tra i diversi livelli.

Per quanto riguarda il lato back-end, il relativo modello di dominio e la descrizione delle scelte di design adottate saranno trattati nella sezione dedicata.

4.1.1 Front-end

Qui di seguito è illustrato il *modello di presentazione* (front-end), evidenziando la struttura gerarchica dei componenti grafici.

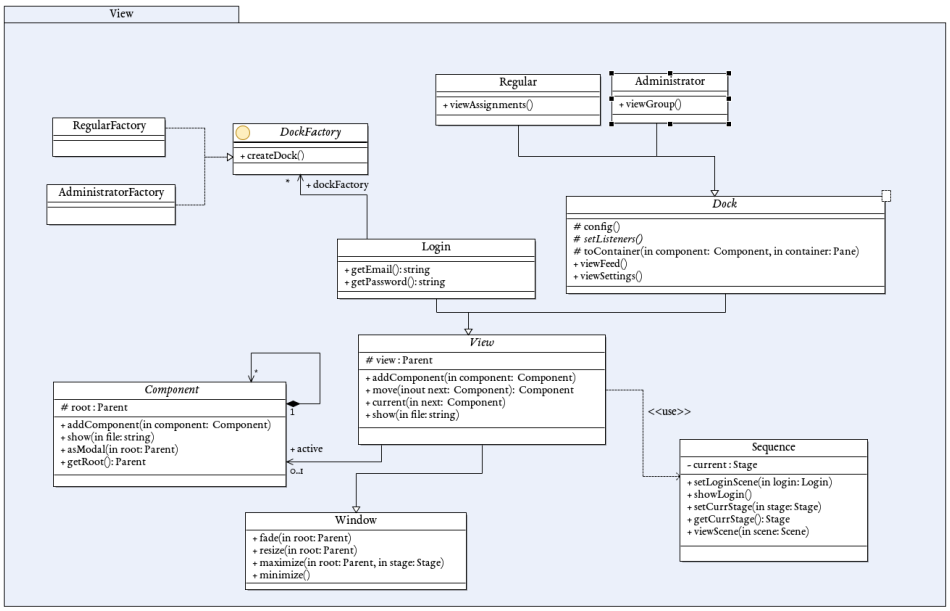


Figura 4.1: Livello View

Sempre in corrispondenza del livello *View* si trovano le seguenti classi che si estendono solo orizzontalmente secondo le proprietà della composizione ricorsiva.

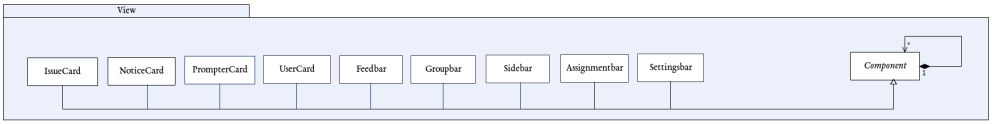


Figura 4.2: Pattern *Composite*

La struttura di questo pattern garantisce un'elevata estendibilità: qualora fosse necessario introdurre una nuova tipologia di componente grafico (es. la pagina della Dashboard), distinta da quelle già presenti nel sistema, è sufficiente definire una nuova classe concreta che estenda *Component*. Grazie alla condivisione della classe astratta in comune, il componente grafico acquisisce immediatamente la capacità di inserirsi in qualsiasi punto della catena di composizione a runtime.

Infine, è possibile arricchire l'oggetto base con la nuova funzionalità senza dover modificare né la classe *Component* e né i componenti preesistenti, aderendo rigorosamente al principio Open/Closed.

Un altro vantaggio della composizione ricorsiva risiede nella gestione efficiente del ciclo di vita degli oggetti. Infatti, quando una vista viene rimossa, vengono rilasciati anche tutti i componenti che la compongono, semplificando enormente il lavoro da parte del Garbage Collector.

Le classi `Feedbar`, `Groupbar`, `Sidebar`, `Assignmentbar` e `Settingsbar` sono componenti principali: contengono altri componenti, come le card e i moduli, secondo la composizione ricorsiva di `Component`. Sono loro a inserire nel proprio contenitore i componenti che il livello `Presenter` restituisce già popolati con i dati provenienti dal back-end. La struttura uniforme del pattern permette inoltre che uno stesso componente sia, a seconda del punto dell'albero in cui si trova, contenitore di altri componenti oppure foglia.

È bene notare, che i componenti grafici sono stati rappresentati da classi spogliate sia dei metodi sia degli attributi. Il fine ultimo del diagramma è comunicare come il pattern *Composite* organizzi i componenti grafici all'interno di un albero, il quale riflette altresì la composizione gerarchica dei componenti descritti dai file FXML.

Per tale motivo, per evitare di appesantire il diagramma, alcune classi sono state omesse; approfittando del grande vantaggio di seguire lo stesso schema di quelle riportate, la loro aggiunta non avrebbe aumentato la comprensione.

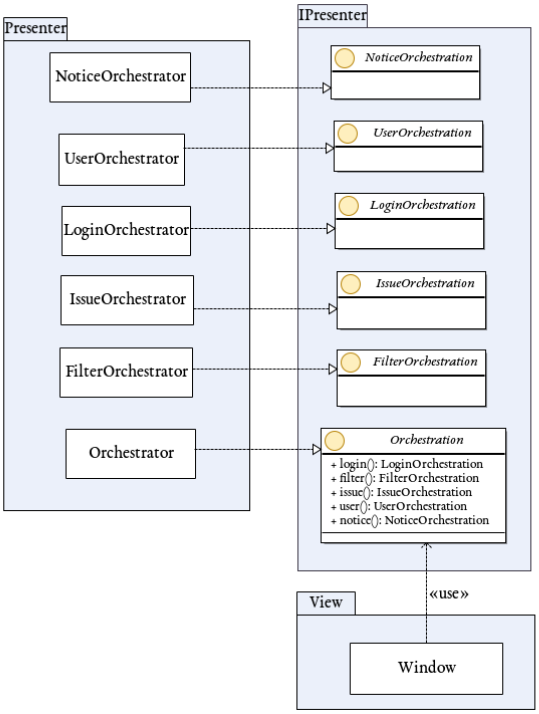


Figura 4.3: Livello View e livello IPresenter

La figura mostra il punto di contatto tra il livello `View` e il livello `IPresenter`. La classe `Window`, radice della gerarchia di `View` e di `Component`, custodisce un oggetto di tipo `Orchestration` che viene istanziato una sola volta all'avvio dell'applicazione.

Tale scelta evita di propagare gli orchestratori lungo la catena dei costruttori che hanno come termine le viste concrete. Inoltre, ogni classe appartenente alla gerarchia di `Window` è in grado di invocare l'orchestratore di cui ha bisogno, senza riferirsi agli oggetti dello strato `Presenter`.

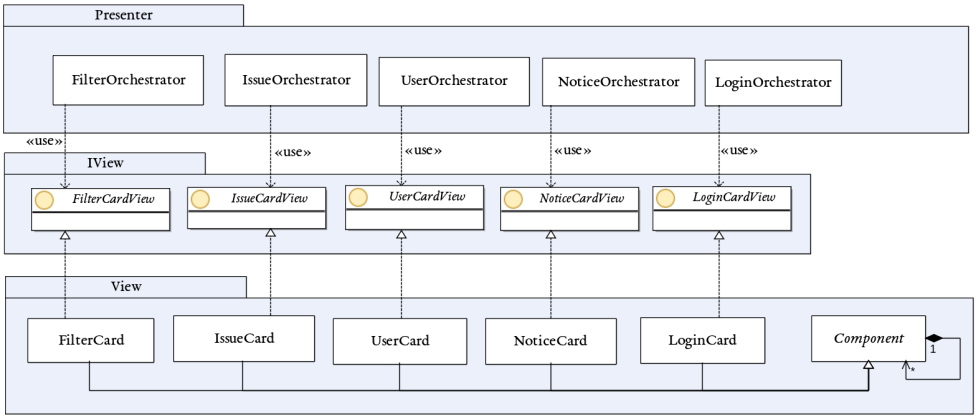


Figura 4.4: Collegamento tra il livello `Presenter`, il livello `IView` e `View`

Questo diagramna chiarisce il motivo di custodire all'interno della classe `Window` un oggetto di tipo `Orchestration`. Ogni card ha bisogno del proprio orchestratore per inviare i dati e riceverli, e la soluzione adottata evita di propagare gli oggetti orchestratori lungo la catena di construttori.

Nel diagramma sono riportate solo le classi a cui il `Presenter` accede tramite le interfacce del livello `IView`. Le altre classi, come `Feedbar` e `Groupbar`, non sono accedute dal `Presenter`: sono loro a invocarne i contratti, attraverso il riferimento a `Orchestration` presente in `Window`, e a ricevere come risultato i dati già composti, come mostrato in Figura 4.3.

La figura mostra anche il meccanismo di comunicazione tra il livello `Presenter` e il livello `IView`. Si prenda, a titolo di esempio, la classe `IssueCard`, che rappresenta sia una issue che si vuole creare sia una issue già esistente nel sistema. Si supponga che classe `Feedbar`, responsabile della visualizzazione e della creazione delle issue, invochi il metodo `create()` di `IssueOrchestrator` tramite il riferimento a `Orchestration`.

Il flusso di esecuzione è il seguente: Come prima operazione, l'utente inserisce i valori relativi alla priorità, al titolo, alla tipologia e così via all'interno della card. L'oggetto con i campi compilati viene passato all'orchestratore che estrapola i campi, invocando i metodi dell'interfaccia, costruisce l'oggetto per il backend e lo invia.

La risposta del back-end viene convertita dal `Mapper` in una `IssueCard` che l'orchestrator restituisce alla `Feedbar` per la visualizzazione. `Assembler` e `Mapper`, che fanno parte del livello `Presenter`, sono descritti nella Figura 4.6.

Il livello `Presenter`, come specificato dal pattern, comunica con il back-end mendiante il livello `Service` che rispecchia i servizi forniti dalla parte back-end. La seguente figura illustra chiaramente questa situazione.

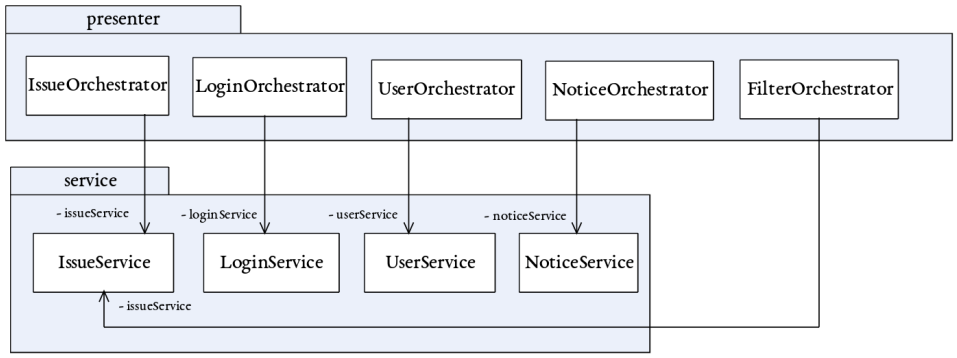


Figura 4.5: Collegamento tra il livello `Presenter` e il livello `Service`

La situazione rappresentata dalla Figura 4.6 descrive due momenti: il momento dell'impacchettamento dei dati per essere inviati al server, e il momento della conversione dei dati ricevuti dal server stesso.

Front-end e Back-end comunicano attraverso un contratto di comunicazione di informazioni, rappresentato dalle classi DTO. Gli orchestratori assemblano i dati provenienti dai componenti grafici in pacchetti di dati in formato DTO. L'oggetto prodotto viene poi passato all'ultimo livello (*Service*) che è il vero e proprio responsabile dell'invio dei dati. I dati ricevuti dal server vengono successivamente convertiti nel componente grafico corretto.

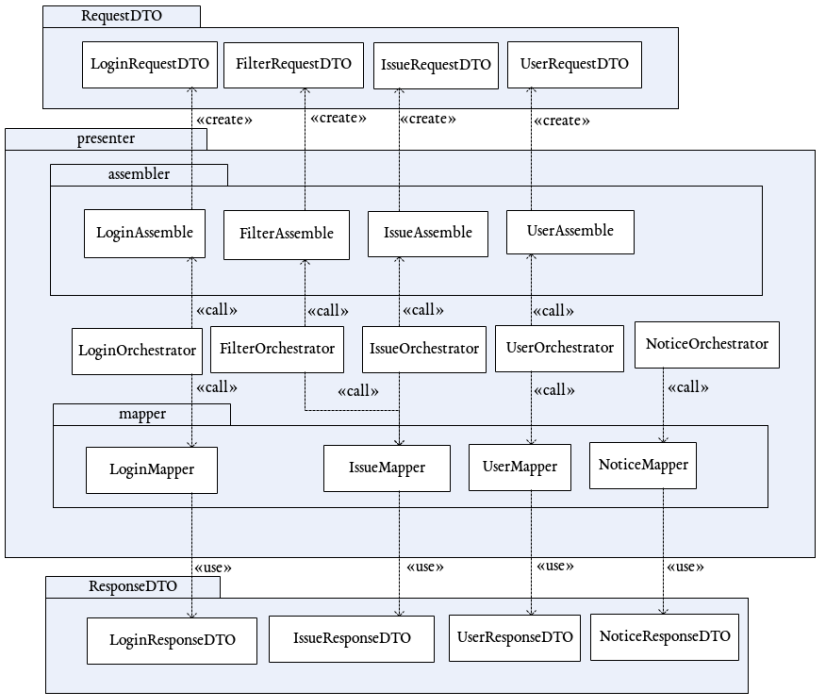


Figura 4.6: Collegamento tra il livello `Presenter` e il livello `Dto`

L'ultima figura mostra l'ultimo punto di contatto dell'architettura. In tale situazione le classi `service` si servono dei contratti di comunicazione che definisco il formato di scambio dati con il back-end. Le classi del package `service` non conoscono il `Presenter` né la `View`: ricevono dall'orchestratore un oggetto di tipo `RequestDTO`, con cui costruiscono la richiesta HTTP da inviare al server.

La risposta del server viene convertita in un oggetto di tipo `ResponseDTO`, che il servizio restituisce all'orchestrator; come descritto nella Figura 1.11, questi lo passa al mapper, che lo converte nel componente grafico corretto.

La dipendenza *«use»* indica che le classi del package `service` usano l'oggetto di tipo `RequestDTO` ricevuto come parametro. La creazione degli oggetti di tipo `ResponseDTO` è espressa invece dalla dipendenza *«create»*.

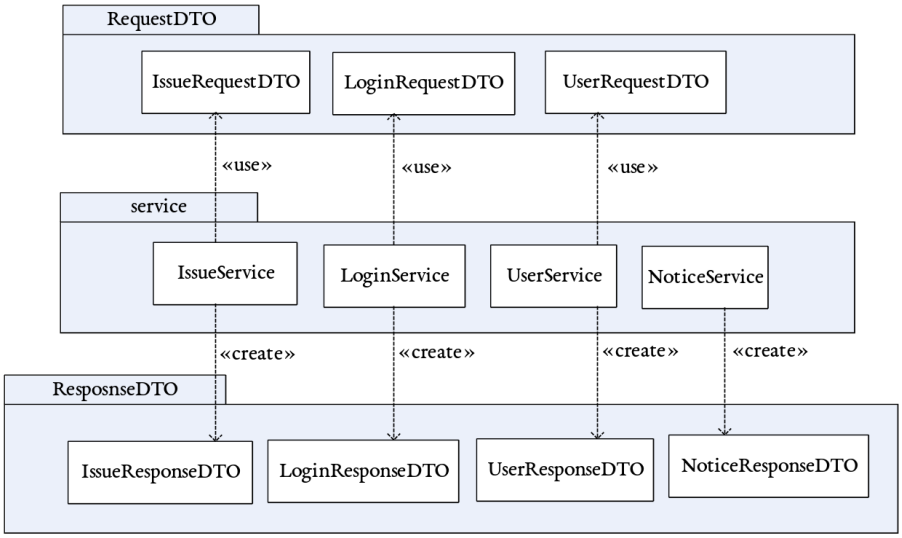


Figura 4.7: Collegamento tra il livello `Service` e il livello `dto`

4.1.2 Back-end

La presente sezione descrive le principali scelte fatte nel progettare il codice del back-end e il motivo di ognuna di esse. Qui di seguito è illustrato il *modello di dominio*.

I ruoli come sottoclassi

I ruoli degli utenti non sono un campo da controllare con degli if, ma sottoclassi di Utente: Amministratore e UtenteNormale (entrambi UtenteInterno) e UtenteEsterno (lo stakeholder, che può solo consultare le issue).

In questo modo i permessi diventano tipi: il creatore di un'issue è di tipo UtenteInterno, l'assegnatario di un bug e il destinatario di una notifica sono di tipo UtenteNormale. È quindi il compilatore a impedire, per esempio, che uno stakeholder risulti autore di un'issue, senza controlli sparsi nel codice.

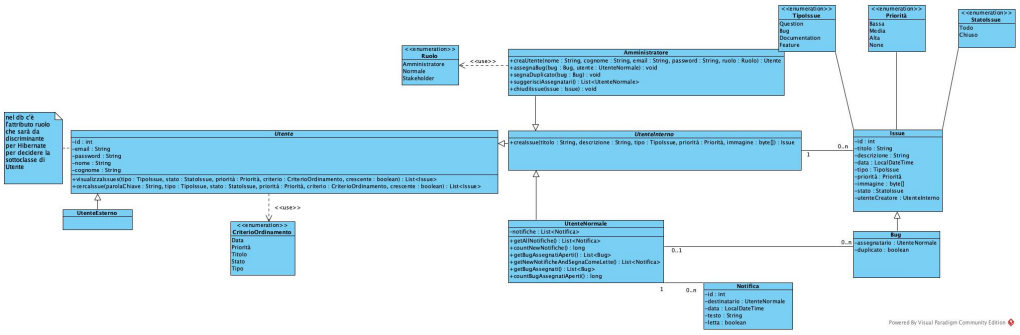


Figura 4.8: Modello di dominio

Bug come sottoclasse di Issue

Solo i bug si possono assegnare e segnare come duplicati, quindi Bug è una sottoclasse di Issue con i suoi campi in più (assegnatario e duplicato).

I metodi assegnaBug e segnaDuplicato ricevono un Bug: chiamarli su una question o una feature non è possibile. Il metodo creaIssue crea un Bug se e solo se il tipo indicato è BUG.

Stato, priorità e duplicati

StatoIssue (TODO, CHIUSO) sta su Issue e non solo su Bug, perché ogni issue nasce TODO e può essere chiusa.

Priorita ha un valore NONE per "priorità non indicata" invece di null: così la priorità è sempre valorizzata, e filtrare o ordinare per priorità non richiede di gestire i valori mancanti.

Il bug duplicato è indicato con un semplice valore vero/falso, senza un riferimento al bug originale: il requisito chiede solo di segnarlo e chiuderlo, e un riferimento in più avrebbe aggiunto vincoli senza una funzionalità che li usasse.

Organizzazione dei service

I service sono divisi in due gruppi:

- Service "attore" (AmministratoreService, UtenteInternoService, UtenteNormaleService, UtenteService): contengono le operazioni del class diagram, cioè le azioni che un utente compie, raggruppate per la classe che le esegue.

- Service "di entità" (IssueService, BugService, NotificaService): contengono le operazioni di base e di supporto, che non sono azioni di un utente ma vengono usate dai service attore.

La regola è: se un utente la compie come azione va nel service attore, se è un effetto di un'altra azione va nel service di entità. Per esempio creaNotifica sta in NotificaService e non in AmministratoreService, perché nessun utente "crea una notifica": la notifica

è un effetto di assegnaBug, segnaDuplicato e chiudiIssue, che la riusano tutti. Così il codice segue il class diagram, e ogni service attore corrisponde a una sua classe.

I service inoltre non controllano i permessi: non ricevono un parametro Amministratore per sapere chi li sta chiamando. I permessi si controllano prima, sugli endpoint (vedi 3c.4). L'utente loggato arriva al service solo quando è un dato dell'operazione, per esempio il creatore di un'issue in creaIssue.

DTO e validazione

I controller non restituiscono mai le entità, ma dei DTO (Data Transfer Object): semplici record Java con i soli campi da mandare al client. Le entità non sono adatte a essere trasformate in JSON:

- Utente contiene l'hash della password, che non deve uscire dal server.

- Alcune relazioni sono circolari (un utente ha le sue notifiche, ogni notifica ha il suo destinatario) e la conversione in JSON non finirebbe mai.

- Il JSON è il contratto con il front-end: con i DTO un cambiamento nel database non lo rompe.

Anche i dati in ingresso arrivano come DTO (per esempio CreaIssueRequest) e vengono controllati su due livelli. I controlli di formato (campi obbligatori, lunghezze massime uguali a quelle delle colonne, formato dell'email, dimensione dell'immagine) sono annotazioni sui DTO, come @NotBlank e @Size, e rifiutano la richiesta prima che arrivi alla logica. Le regole di business (email già in uso, bug già chiuso, bug già duplicato) sono invece nei service.

Sicurezza

L'autenticazione usa Spring Security con la sessione HTTP: dopo il login il server ricorda l'utente e il client riceve un cookie da allegare alle richieste successive. All'inizio avevamo valutato i token JWT, ma li abbiamo scartati perché più complessi da gestire (firma, scadenza, rinnovo) senza vantaggi per un sistema con un solo back-end e un solo client. Le password sono salvate solo come hash BCrypt, e al login un'email inesistente e una password sbagliata danno lo stesso errore, per non rivelare quali email sono registrate.

I permessi sono dichiarati con @PreAuthorize su ogni endpoint (per esempio hasRole('AMMINISTRATORE') per assegnare un bug): l'attore del class diagram diventa il ruolo richiesto dall'endpoint. Tutto tranne il login richiede di essere autenticati, quindi se ci si dimentica un'annotazione l'endpoint resta comunque chiuso a chi non ha fatto l'accesso.

Gestione degli errori

I service segnalano i problemi lanciando eccezioni, e un'unica classe, GlobalExceptionHandler, le trasforma nella risposta HTTP giusta: 400 per dati non validi, 401 per credenziali sbagliate, 404 per una risorsa inesistente, 409 per un'operazione in conflitto con lo stato attuale (per esempio assegnare un bug chiuso). Gli errori 401 e 403 generati da Spring Security usano lo stesso formato. In questo modo i controller non hanno try/catch e il client riceve sempre errori con la stessa struttura, con un messaggio che può mostrare all'utente.

Regole di business

Dove i requisiti lasciavano margine abbiamo deciso noi il comportamento:

- creaUtente: l'email viene salvata in minuscolo e deve essere unica, per evitare due account con la stessa email scritta in modo diverso.

- assegnaBug: un bug chiuso non si può assegnare. Se il bug passa da un utente a un altro, ricevono la notifica sia il nuovo sia il vecchio assegnatario.

- suggerisciAssegnatari: gli utenti sono ordinati per numero di bug ancora aperti (TODO) assegnati, dal meno al più carico.

- segnaDuplicato: un bug già segnato come duplicato non si può segnare di nuovo; il bug viene chiuso e l'assegnatario avvisato.

- chiudiIssue: è riservata all'amministratore, e chiudere un'issue già chiusa non dà errore.

Le operazioni che fanno più scritture insieme, come assegnaBug (aggiorna il bug e crea le notifiche), sono @Transactional: o riescono tutte o nessuna, così non restano notifiche di un'assegnazione mai avvenuta.

Filtri e ordinamenti nel database

La vista delle issue permette di filtrare e ordinare per vari criteri. Invece di caricare tutte le issue e ordinarle in Java, abbiamo scritto le query in IssueRepository: una parte comune con i filtri e un ORDER BY diverso per ogni criterio e direzione. Il service sceglie la query giusta con uno switch. Così è il database, che lo fa in modo efficiente, a filtrare e ordinare, e al back-end arrivano solo le issue richieste.

4.2 Schema di persistenza dei dati

In questa sezione descriviamo come Bugboard26 conserva i propri dati: quale database abbiamo scelto, come è fatto lo schema, come le classi del class diagram sono state trasformate in tabelle e come lo schema viene creato e aggiornato nel tempo.

4.2.1 Database e gestione dello schema

Per la persistenza abbiamo usato MySQL 8, un database relazionale. I dati di Bugboard26 sono strutturati e collegati tra loro (un'issue ha un autore, un bug ha un assegnatario, una notifica ha un destinatario), quindi il modello relazionale, con le sue chiavi esterne e le transazioni, è quello più adatto.

Lo schema è stato progettato con MySQL Workbench, disegnando prima il modello ER e generando poi automaticamente lo script SQL di creazione delle tabelle (forward engineering).

Il back-end accede al database tramite JPA (con Hibernate come implementazione) e Spring Data JPA: le classi del dominio sono entità mappate sulle tabelle, e i repository offrono ai service le operazioni di lettura e scrittura. Hibernate però non crea le tabelle: è configurato in modalità validate, cioè all'avvio controlla soltanto che le entità Java corrispondano allo schema, e se non corrispondono l'applicazione non parte. Lo schema lo crea e lo aggiorna Flyway (vedi paragrafo 3.7). In questo modo è il database a comandare e il codice si adegua a lui, e non il contrario.

4.2.2 Schema ER

Lo schema è composto da quattro tabelle: Utenti, Issues, Bugs e Notifiche.

La chiave gialla indica la chiave primaria, il rombo rosso una chiave esterna. - Il rombo pieno indica una colonna obbligatoria (NOT NULL), quello vuoto una colonna facoltativa. - La linea tratteggiata indica una relazione non identificante, la linea continua una relazione identificante, in cui la chiave esterna ùfa parte della chiave primaria (è il caso di Bugs).

4.3 Report di qualità del codice

Lo strumento utilizzato per assicurare un ottimo livello di qualità del codice del back-end è stato SonarQube for IDE, plugin per IntelliJ. Sono stati risolti tutti i warning e i code smells durante lo sviluppo del progetto, e per il report finale è stato usato SonaeQube Cloud: qui di seguito il summary indicante due errori relativi alla sicurezza, ma che sono in realtà scelte deliberate.

Data la natura del progetto si è optato di creare il primo utente admin direttamente dal codice, per ovviare a una moltitudine di problemi di testing e di demo su diverse macchine. Tale scelta però comporta la scrittura del nome utente ed ovviamente della password (seppur sempre in formato hash) in maniera hardcoded, e ciò rappresenta una falla di sicurezza. Nel momento della messa in produzione del software una nuova migration Flyway risolve tutto.

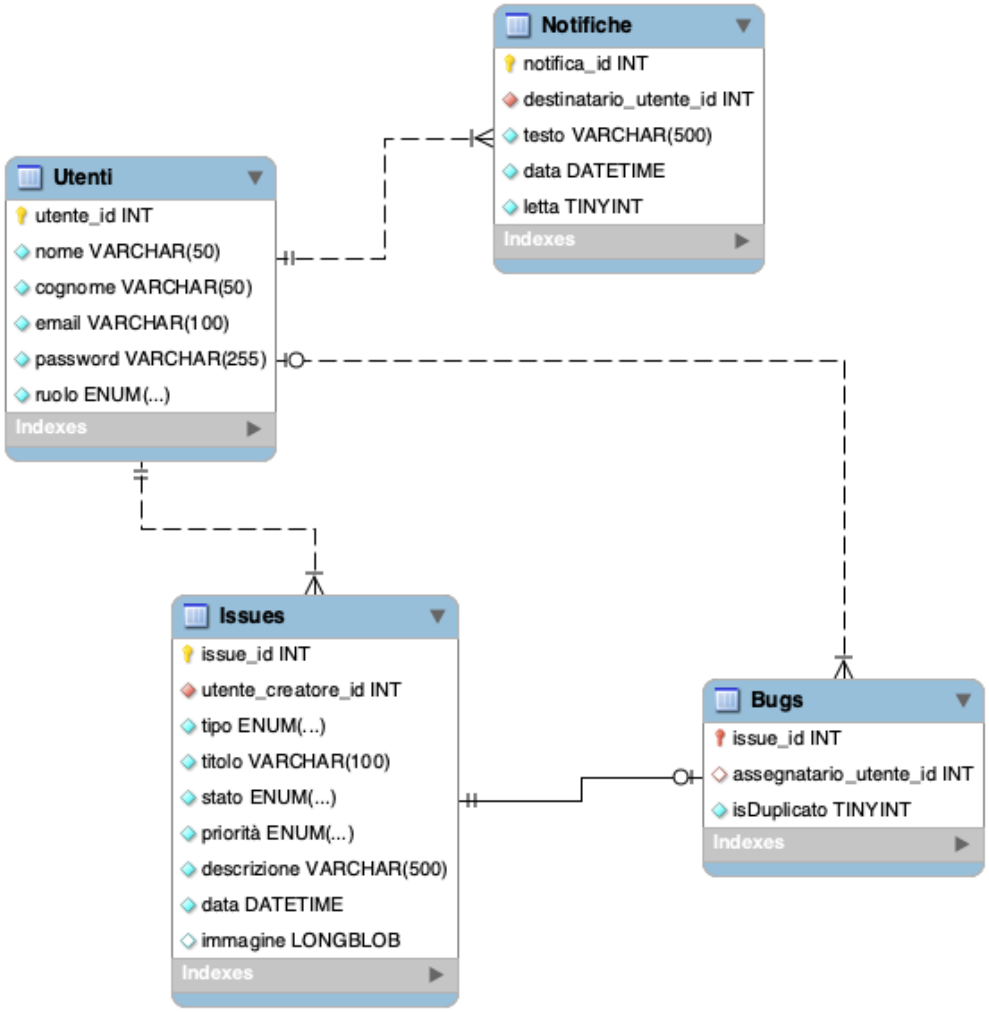


Figura 4.9: Schema ER

4.3.1 Make sure disabling Spring Security's CSRF protection is safe

Questa problematica indica che la protezione CSRF (Cross-Site Request Forgery) di Spring Security è stata disattivata. Tale scelta è stata presa in quanto il CSRF è un attacco che sfrutta l'invio automatico dei cookie da parte del browser, ma il nostro client è un'applicazione JavaFX che gestisce da sé il cookie di sessione, quindi l'attacco non è possibile e la protezione è stata disattivata. Se in futuro si aggiungesse un front-end web, la protezione andrebbe riattivata.

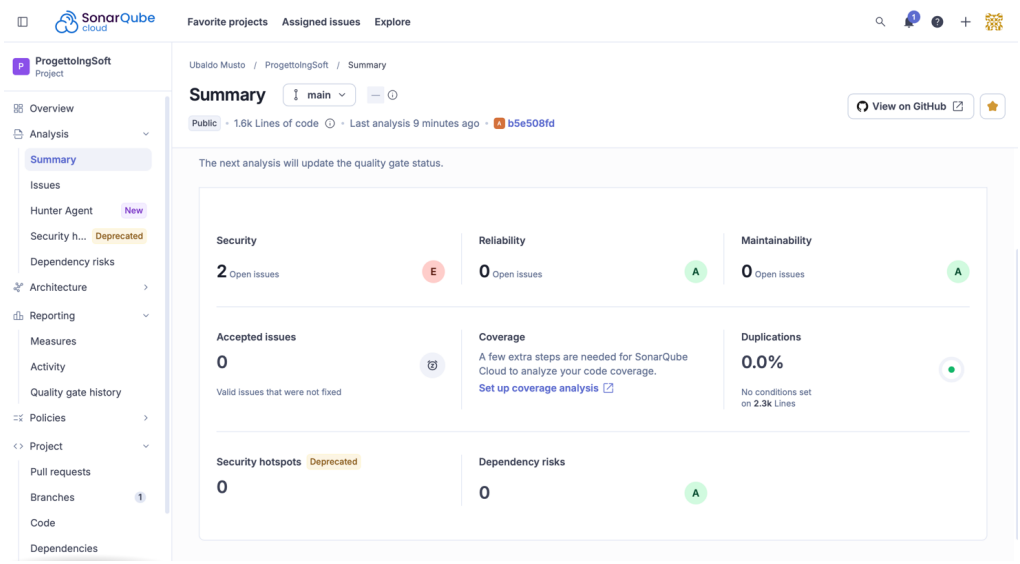


Figura 4.10: Report

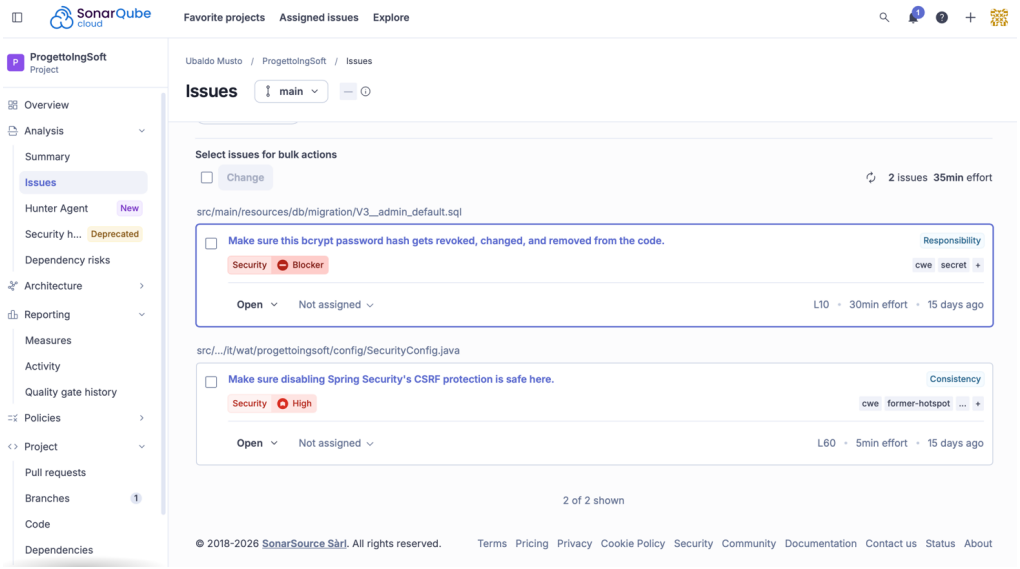


Figura 4.11: Report

Capitolo 5

Verifica e convalida

5.1 Progettazione dei casi di test

Questa sezione descrive nel dettaglio la progettazione dei casi di test e le motivazioni delle strategie adottate. L'obiettivo è definire un insieme di casi di test efficace, in grado di verificare che il sistema soddisfi i requisiti.

Tra i diversi approcci alla progettazione dei test, la strategia adottata si basa sul partizionamento in classi di equivalenza. Il dominio dei dati in ingresso viene suddiviso in classi, ciascuna composta da valori per cui il sistema dovrebbe comportarsi in modo equivalente, e per ogni classe si sceglie almeno un caso di test rappresentativo. Questa scelta riduce il numero di test necessari senza rinunciare alla copertura dei comportamenti distinti del sistema.

5.2 Descrizione dei casi di testi in JUnit

La presente sezione si occuperà di rappresentare i casi di test definiti per esaminare il comportamento delle funzionalità più complesse e per accertarne il corretto funzionamento. A questo scopo, è stato utilizzato il framework JUnit 5.

Per facilitare la leggibilità, i casi di test sono stati suddivisi in base al sottosistema di appartenenza. I primi casi di test che verranno illustrati sono quelli definiti nella parte front-end; successivamente verranno descritti quelli della parte back-end.

I metodi selezionati (lato front-end) per gli Unit Test sono i seguenti:

1. `selectToggleFilter()`: imposta un filtro di ricerca a partire dai campi impostati nell'interfaccia grafica. Lo scopo è verificare che ogni combinazione di scelte, compresa l'assenza di selezione, produca un filtro valido.
2. `setDate()`: traduce una data in formato UTC in una data leggibile dall'utente. Lo scopo è verificare che le date significative (passaggio di mezzanotte o di fine anno, valore mancante o malformato) siano convertite correttamente.

Per la definizione delle classi di equivalenza è stato adottato l'*approccio basato sulla funzionalità*, in cui il processo di partizionamento si basa rigorosamente sui requisiti e non sull'implementazione del codice.

Le classi di equivalenza sono pensate in relazione al comportamento del metodo. Quindi i parametri non sono stati valutati in modo isolato, ma si è tenuto conto del contesto e delle relazioni logiche esistenti tra di essi.

5.2.1 Metodo `selectToggleFilter()`

La firma del metodo è la seguente:

```
void selectToggleFilter(ToggleGroup group, String filterValue)
```

`filterValue` rappresenta la tipologia di filtro selezionata appartenente ad uno specifico gruppo, il quale è stato indicato nel primo parametro, `group`.

Per il parametro `group` il dominio è $G = \{\text{typologyGroup}, \text{stateGroup}, \text{priorityGroup}\}$ I gruppi vengono creati nativamente dall'interfaccia tramite FXML, venendo già popolati con i rispettivi `RadioButton`. Inoltre, il riferimento non è mai nullo e si ottiene la seguente partizione

$$C_1 = \{\text{typologyGroup}\}, C_2 = \{\text{stateGroup}\}, C_3 = \{\text{priorityGroup}\}$$

Per il parametro `filterValue`, sia $S = \{w | w \text{ rappresenta una stringa}\} \cup \{NIL\}$ l'insieme di tutte le stringhe incluso il valore nullo, sono state identificate le seguenti classi di equivalenza

$$C_4 = \{\text{BUG}, \text{FEATURE}, \text{DOCUMENTATION}, \text{QUESTION}\}$$

$$C_5 = \{\text{TODO}, \text{CHIUSO}\}, C_6 = \{\text{ALTA}, \text{MEDIA}, \text{BASSA}\}$$

$$C_7 = S \setminus \bigcup_{i=4}^{6} C_i$$

Secondo l'implementazione del metodo `selectToggleFilter`, ogni classe è costituita da valori validi. Per quanto riguarda la classe C_7, se la stringa non corrisponde a nessun `UserData` dei `RadioButton` definiti, il metodo lascia il gruppo senza selezione.

Il criterio di copertura adottato *Each Choice Coverage*, che ha permesso di generare la seguente batteria di test

Tabella 5.1: Casi di test

Caso	`group`	`filterValue`	Classi	Esito atteso
TC1	`typologyGroup`	`BUG`	C_1, C_4	pulsante BUG selezionato
TC2	`priorityGroup`	`ALTA`	C_3, C_6	pulsante ALTA selezionato
TC3	`stateGroup`	`TODO`	C_2, C_5	pulsante TODO selezionato
TC4	`stateGroup`	`BLOCCATO`	C_2, C_7	nessuna selezione

```
@Test
    void testFilterTypologyValid()  {
        FilterFake view = new FilterFake();
        ToggleGroup typologyGroup = new ToggleGroup();

        view.toggle("BUG",
                typologyGroup);
        view.toggle("FEATURE",
                typologyGroup);
        view.toggle("DOCUMENTATION",
                typologyGroup);
        view.toggle("QUESTION",
                typologyGroup);

        view.selectToggleGroup(typologyGroup,
                "BUG");

        assertNotNull(typologyGroup
                .getSelectedToggle());
        assertEquals("BUG", typologyGroup
                .getSelectedToggle()
                .getUserData());
    }

    @Test
    void testFilterPriorityValid()  {
        FilterFake view = new FilterFake();
        ToggleGroup priorityGroup = new ToggleGroup();

        view.toggle("ALTA",
                priorityGroup);
        view.toggle("MEDIA",
                priorityGroup);
        view.toggle("BASSA",
                priorityGroup);

        view.selectToggleGroup(priorityGroup,
                "ALTA");

        assertNotNull(priorityGroup
                .getSelectedToggle());
        assertEquals("ALTA", priorityGroup
                .getSelectedToggle()
                .getUserData());
    }
```

```
@Test
void testFilterStateValid()  {
    FilterFake view = new FilterFake();
    ToggleGroup stateGroup = new ToggleGroup();

    view.toggle("TODO",
            stateGroup);
    view.toggle("CHIUSO",
            stateGroup);

    view.selectToggleGroup(stateGroup,
            "TODO");

    assertNotNull(stateGroup
            .getSelectedToggle());
    assertEquals("TODO", stateGroup
            .getSelectedToggle()
            .getUserData());
}

@Test
void testFilterBadValid()  {
    FilterFake view = new FilterFake();
    ToggleGroup stateGroup = new ToggleGroup();

    view.toggle("TODO",
            stateGroup);
    view.toggle("CHIUSO",
            stateGroup);

    view.selectToggleGroup(stateGroup,
            "APERTO");
    assertNull(stateGroup
            .getSelectedToggle());
}
```

5.2.2 Metodo `setDate()`

Poiché non sono presenti metodi pubblici non banali con due o più parametri che non effettuino chiamate verso il back-end, si è scelto di testare uno dei metodi ritenuti più complessi, setDate, pur accettando un solo parametro.

La firma del metodo è la seguente:

```
void setDate(LocalDateTime date)
```

Il metodo presenta un dominio di input molto ampio. In base al comportamento atteso, in cui ad esempio intervalli inferiori a un minuto vengono tutti indicati come "*Just now*", è stata ricavata la seguente partizione:

$$C_1 = \{NIL\}\,, C_2 = \{w \mid w \text{ rappresenta una data del futuro}\}\,,$$

$$C_3 = \{\Delta \mid 0 \leq \Delta < 60\text{s}\}$$

dove Δ rappresenta il tempo trascorso tra la data fornita e il tempo attuale.

$$C_4 = \{\Delta \mid 0 < \Delta < 59\text{min}\}, C_5 = \{\Delta \mid 0 < \Delta < 24\text{h}\}, C_6 = \{\Delta \mid 0 < \Delta < 7\text{d}\}$$

$$C_7 = \{\Delta \mid 7d \leq \Delta < 1\text{m}\}, C_8 = \{\Delta \mid 0 < \Delta < 12\text{m}\},$$

$$C_9 = \{\Delta \mid \Delta > 0\}$$

dove 1m rappresenta 1 mese, 7d rappresenta 7 giorni e l'ultima classe di equivalenza esprime l'intervallo in anni.

Il criterio di copertura adottato Each Choice Coverage, che ha permesso di generare la seguente batteria di test.

```
@Test
void testDateNull()  {
    assertThrows(NullPointerException.class,
            () -> {
        NoticeDate notice =
                new NoticeDate();
        notice.setDate(null);
    });
}

@Test
void testPastDateYear()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusYears(3));

    assertNotNull(view);
    assertEquals("3 years ago", view
            .getDate());
}

@Test
void testPastDateMonth()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusMonths(11));

    assertNotNull(view);
    assertEquals("11 months ago", view
            .getDate());
}
```

```
@Test
void testPastDateWeek()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusDays(14));

    assertNotNull(view);
    assertEquals("2 weeks ago", view
            .getDate());
}

@Test
void testPastDateDay()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusDays(6));

    assertNotNull(view);
    assertEquals("6 days ago", view
            .getDate());
}

@Test
void testPastDateHour()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusHours(23));

    assertNotNull(view);
    assertEquals("23 hours ago", view
            .getDate());
}

@Test
void testPastDateMinute()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusMinutes(59));

    assertNotNull(view);
    assertEquals("59 minutes ago", view
            .getDate());
}

@Test
void testPastDateSecond()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .minusSeconds(5));

    assertNotNull(view);
    assertEquals("Just now", view
            .getDate());
}
```

```
@Test
void testFutureDate()  {
    NoticeDate view = new NoticeDate();
    view.setDate(LocalDateTime
            .now(ZoneOffset.UTC)
            .plusMonths(5));

    assertNotNull(view);
    assertEquals("Just now", view
            .getDate());
}
```

Tabella 5.2: Casi di test

Caso	`date`	Classi	Esito atteso
TC1	`null`	C_1	`NullPointerException`
TC2	`now() - 3 anni`	C_9	"3 years ago"
TC3	`now() - 11 mesi`	C_8	"11 months ago"
TC4	`now() - 2 settimane`	C_7	"2 weeks ago"
TC5	`now() - 6 giorni`	C_6	"6 days ago"
TC6	`now() - 23 ore`	C_5	"23 hours ago"
TC7	`now() - 59 minuti`	C_4	"59 minutes ago"
TC8	`now() - 5 secondi`	C_3	"Just now"
TC9	`now() + 5 secondi`	C_2	"Just now"

5.3 Testing lato back-end

5.3.1 Test plan: creazione di una issue

Obiettivo

Verificare che un utente interno (amministratore o utente normale) possa segnalare un'issue e che il sistema applichi le seguenti regole di servizio: titolo e descrizione obbligatori, stato iniziale TODO, priorità NONE se non indicata e creazione di un oggetto bug solo per le issue di tipo BUG. Verificare inoltre che gli utenti non autorizzati vengano respinti.

Attori del test

- L'endpoint POST /api/issues (che corrisponde a IssueController.creaIssue), che controlla il ruolo dell'utente e la validità dei dati ricevuti. - La classe CreaIssueRequest, che contiene i vincoli sui dati in ingresso (campi obbligatori e lunghezze). - Il metodo UtenteInternoService.creaIssue, che applica le regole di creazione.

Ambiente di testing

Back-end e database MySQL avviati ed operativi; nel database sono presenti l'amministratore di default, un utente normale e uno stakeholder.

Prima di ogni caso di test l'utente indicato ha effettuato il login e la richiesta contiene il suo cookie di sessione.

Tranne dove indicato, il corpo della richiesta è sempre:

"titolo": "Login non funziona", "descrizione": "Errore 500 al login", "tipo": "BUG", "priorita": "ALTA", "immagine": null

Id	*Precondizioni*	*Input*	*Output effettivo*	*Postcondizioni*
TP1	Nessun login	Corpo di riferimento	401 Unauthorized, “Utente non autenticato”	Nessuna issue creata
TP2	Login come stakeholder	Corpo di riferimento	403 Forbidden, ruolo non ammesso, “Operazione non permessa”	Nessuna issue creata
TP3	Login come utente normale	Corpo di riferimento	201 Created	Bug creato (nel db righe in Issues e in Bugs). UtenteCreatore = l’utente loggato
TP4	Login come amministratore	Corpo di riferimento	201	Bug creato (nel db righe in Issues e in Bugs). UtenteCreatore = l’utente loggato
TP5	Login come utente normale	tipo = FEATURE	201	Riga creata solo in Issues
TP6	Login come utente normale	priorità assente	201, priorità NONE	Issue creata
TP7	Login come utente normale	titolo assente	400 Bad request, richiesta non valida, “Il titolo è obbligatorio”	Nessuna issue creata
TP8	Login come utente normale	titolo di 1 carattere	201	Issue creata
TP9	Login come utente normale	titolo di 100 caratteri	201	Issue creata
TP10	Login come utente normale	titolo di 101 caratteri	400 , “Il titolo può avere al massimo 100 caratteri”	Nessuna issue creata
TP11	Login come utente normale	descrizione di 500 caratteri	201	Issue creata
TP12	Login come utente normale	descrizione di 501 caratteri	400, “La descrizione può avere al massimo 500 caratteri”	Nessuna issue creata
TP13	Login come utente normale	tipo assente	400, “Il tipo è obbligatorio”	Nessuna issue creata
TP14	Login come utente normale	tipo = "PIPPO" oppure priorità = "URGENTE"	400	Nessuna issue creata
TP15	Login come utente normale	immagine di 2 MB	201, con immagine	Issue creata
TP16	Login come utente normale	JSON non valido	400, “Richiesta non valida”	Nessuna issue creata

Criteri di superamento

Il test plan è superato se tutti i casi di test producono l'output e le postcondizioni attese.

5.3.2 Metodo `creaIssue()`

La firma del metodo è la seguente:

```
creaIssue(idCreatore, titolo, descrizione, tipo, priorita, immagine)
```

Da questo metodo abbiamo identificato le seguenti classi di equivalenza:

idCreatore: CE1 = $\{x$ è l'id di un utente interno esistente$\}$ (Valida)
idCreatore: CE2 = $\{x$ non corrisponde a nessun utente$\}$ (Non valida)
tipo: CE3 = $\{x =$ BUG$\}$ (Valida)
tipo: CE4 = $\{x \in \{QUESTION, DOCUMENTATION, FEATURE\}\}$ (Valida)
priorita: CE5 = $\{x \in \{BASSA, MEDIA, ALTA\}\}$ (Valida)
priorita: CE6 = $\{x$ è NULL$\}$ (Valida, la priorità diventa NONE)
immagine: CE7 = $\{x$ è un array di byte$\}$ (Valida)
immagine: CE8 = $\{x$ è NULL$\}$ (Valida, issue senza immagine)
titolo: CE9 = $\{x$ è una stringa non vuota di al massimo 100 caratteri$\}$ (Valida)
descrizione: CE10 = $\{x$ è una stringa non vuota di al massimo 500 caratteri$\}$ (Valida)

Le classi non valide di titolo e descrizione ($\{x$ è NULL oppure ""$\}$ e
$\{x$ supera la lunghezza massima$\}$) non sono state considerate: la validazione della richiesta le rifiuta con errore 400 prima che creaIssue venga chiamato.

```
class UtenteInternoServiceTest {

    private final Integer idUtenteCreatore = 7;

    private IssueRepository issueRepository;
    private UtenteInternoRepository utenteInternoRepository;
    private UtenteInternoService utenteInternoService;
    private UtenteNormale utenteCreatore;

    // Setup: crea i mock per le repository e la classe contenente
    // creaIssue, infine crea un utente con id=7
    @BeforeEach
    void setUp() {
        issueRepository = mock(IssueRepository.class);
        utenteInternoRepository = mock(UtenteInternoRepository.class);
        utenteInternoService = new UtenteInternoService(issueRepository,
        utenteInternoRepository);

        utenteCreatore = new UtenteNormale();
        utenteCreatore.setId(idUtenteCreatore);
        when(utenteInternoRepository.findById(idUtenteCreatore))
        .thenReturn(Optional.of(utenteCreatore));
    }
```

```
// CI1 - Utente creatore inesistente: la repository non
    // lo trova e lancia l'eccezione
    @Test
    void creaIssue_creatoreNonTrovato_lanciaEccezione() {
        // precondizione: l'id 99 non corrisponde a nessun utente
        when(utenteInternoRepository.findById(99))
        .thenReturn(Optional.empty());

        // si lancia l'eccezione con questo id
        assertThrows(EntityNotFoundException.class, () ->
                        utenteInternoService
                        .creaIssue(99, "Titolo", "Descrizione",
                                TipoIssue.BUG, Priorita.ALTA, null));

        // controlla che nessuna issue è stata salvata
        verify(issueRepository, times(0)).save(any());
    }

    // CI2 - Issue tipo BUG: viene creato un Bug, non
    // assegnato e non duplicato.
    @Test
    void creaIssue_tipoBug_creaBug() {

        utenteInternoService.creaIssue(idUtenteCreatore, "Login non funziona",
        "Errore 500 al login",
                TipoIssue.BUG, Priorita.ALTA, null);

        // assert
        verify(issueRepository).save(any(Bug.class));
        verify(issueRepository).save(argThat(allOf(
                hasProperty("tipo", equalTo(TipoIssue.BUG)),
                hasProperty("assegnatario", is(nullValue())),
                hasProperty("duplicato", is(false)))));
    }

    // CI3 - Issue tipo QUESTION
    @Test
    void creaIssue_tipoQuestion() {

        utenteInternoService.creaIssue(idUtenteCreatore, "Avete provato
        la nuova funzione dell'IDE?",
                "Vorrei qualche delucidazione in merito",
                TipoIssue.QUESTION, Priorita.BASSA, null);

        // assert: salvata un'issue di tipo QUESTION
        verify(issueRepository).save(argThat(hasProperty("tipo",
        equalTo(TipoIssue.QUESTION))));
    }
```

```
// CI4 - Issue tipo DOCUMENTATION
    @Test
    void creaIssue_tipoDocumentation() {

        utenteInternoService.creaIssue(idUtenteCreatore, "Cosa restituisce
                questo endpoint?",
                "Manca una documentazione adeguata",
                TipoIssue.DOCUMENTATION, Priorita.MEDIA, null);

        // assert
        verify(issueRepository).save(argThat(hasProperty("tipo",
        sequalTo(TipoIssue.DOCUMENTATION))));
    }

 // CI5 - Issue tipo FEATURE
    @Test
    void creaIssue_tipoFeature() {

        utenteInternoService.creaIssue(idUtenteCreatore,
                "Dimenticato di implementare la crittografia dei dati",
                "Feature importante richiesta dal cliente",
                TipoIssue.FEATURE, Priorita.ALTA, null);

        // assert
        verify(issueRepository).save(argThat(hasProperty("tipo",
        equalTo(TipoIssue.FEATURE))));
    }

    // CI6 - Issue con priorità non indicata: vale NONE.
    @Test
    void creaIssue_prioritaNull() {

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.FEATURE, null, null);

        // assert
        verify(issueRepository)
        .save(argThat(hasProperty("priorita",
        equalTo(Priorita.NONE))));
    }

    // CI7 - Issue con priorità indicata
    @Test
    void creaIssue_prioritaIndicata() {
        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona",
        "Errore 500 al login",
                TipoIssue.FEATURE, Priorita.MEDIA, null);

        // assert
        verify(issueRepository)
        .save(argThat(hasProperty("priorita",
        equalTo(Priorita.MEDIA))));
    }
```

```
    // CI8 - Regole di servizio valide per ogni issue: imposta data
    // di creazione e stato=TODO
    @Test
    void creaIssue_impostaStatoTodoEData() {

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.QUESTION, Priorita.ALTA, null);

        // assert
        verify(issueRepository).save(argThat(allOf(
                hasProperty("stato", equalTo(StatoIssue.TODO)),
                hasProperty("data", is(not(nullValue()))),
                hasProperty("utenteCreatore", equalTo(utenteCreatore)),
                hasProperty("titolo", equalTo("Login non funziona")),
                hasProperty("descrizione", equalTo("Errore 500 al login")),
                hasProperty("immagine", is(nullValue())))));
    }

  // CI9 - Issue con immagine presente
    @Test
    void creaIssue_conImmagine() {
        byte[] immagine = {1, 2, 3, 4};

        utenteInternoService.creaIssue(idUtenteCreatore,
        "Login non funziona", "Errore 500 al login",
                TipoIssue.BUG, Priorita.ALTA, immagine);

        // assert
        verify(issueRepository).save(argThat(hasProperty("immagine",
         equalTo(immagine))));
    }
}
```

CI1 (riga 57): creaIssue(99, "Titolo", "Descrizione", BUG, ALTA, NIL), copre CE2, CE3, CE5, CE8, CE9, CE10
CI2 (riga 72): creaIssue(7, "Login non funziona", "Errore 500 al login", BUG, ALTA, NIL), copre CE1, CE3, CE5, CE8, CE9, CE10
CI3 (riga 87): creaIssue(7, "Avete provato la nuova funzione dell'IDE?", "Vorrei qualche delucidazione in merito", QUESTION, BASSA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI4 (riga 99): creaIssue(7, "Cosa restituisce questo endpoint?", "Manca una documentazione adeguata", DOCUMENTATION, MEDIA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI5 (riga 111): creaIssue(7, "Dimenticato di implementare la crittografia dei dati", "Feature importante richiesta dal cliente", FEATURE, ALTA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI6 (riga 123): creaIssue(7, "Login non funziona", "Errore 500 al login", FEATURE, NIL, NIL), copre CE1, CE4, CE6, CE8, CE9, CE10

CI7 (riga 134): creaIssue(7, "Login non funziona", "Errore 500 al login", FEATURE, MEDIA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI8 (riga 144): creaIssue(7, "Login non funziona", "Errore 500 al login", QUESTION, ALTA, NIL), copre CE1, CE4, CE5, CE8, CE9, CE10
CI9 (riga 161): creaIssue(7, "Login non funziona", "Errore 500 al login", BUG, ALTA, 1, 2, 3, 4), copre CE1, CE3, CE5, CE7, CE9, CE10

Criterio di copertura delle classi di equivalenza

Come criterio di copertura si è adottato l'Each Choice Coverage (ECC): ogni classe di equivalenza di ogni parametro è rappresentata da almeno un caso di test. I test da CI1 a CI9 coprono così tutte le classi individuate.